

Reto 5 – Seguridad y Control de Acceso con JWT

Contexto

Este reto continúa el desarrollo del **sistema de onboarding y offboarding de empleados**. Hasta ahora, se ha construido un sistema distribuido con comunicación sincrónica y asincrónica. Sin embargo, todos los endpoints están "abiertos"; cualquier cliente que conozca la dirección IP y el puerto puede crear o eliminar empleados sin ninguna restricción.

En un entorno del mundo real, la seguridad es fundamental. Por ello, en este cuarto reto se incorporará autenticación y autorización utilizando el estándar **JSON Web Token (JWT)**.

Objetivo

Implementar un mecanismo de seguridad centralizado para el ecosistema de microservicios. Al finalizar este reto, el sistema debe ser capaz de autenticar a los usuarios a través de un nuevo servicio de identidad, emitir tokens JWT y validar dichos tokens en cada petición para proteger el acceso a los recursos existentes.

Requisitos generales

La solución desarrollada debe cumplir con los siguientes requisitos:

- Implementar un nuevo microservicio **Servicio de Autenticación (auth-service)** encargado de verificar credenciales y generar el JWT.
- Proteger los endpoints de los servicios existentes (Empleados, Departamentos, Perfiles, Notificaciones, Vacaciones) validando el JWT.
- El token JWT debe incluir **claims (reclamaciones)** específicas, como el rol de usuario, para implementar autorización.

- Implementar **Control de Acceso Basado en Roles (RBAC)**: ciertos endpoints solo deben ser accesibles para el rol `ADMIN` , mientras que otros pueden ser consumidos por el rol `USER` .
- Implementar el **ciclo de vida completo de la cuenta dirigido por eventos**: alta, desactivación temporal por vacaciones, reactivación y baja permanente.
- Implementar el **scheduler de vacaciones** en el `vacaciones-service` , que dispara los eventos de inicio y fin de período al llegar la fecha.
- Documentar cómo obtener el token y realizar peticiones autenticadas utilizando **OpenAPI (Swagger)**.

Todos los eventos de este reto están definidos en el [Catálogo de Eventos](#). No invente nombres ni cargas útiles.

1. Servicio de Autenticación (`auth-service`)

Este nuevo servicio actuará como el proveedor de identidad del sistema.

Endpoints requeridos

Método	Ruta	Descripción
POST	<code>/auth/login</code>	Recibe credenciales (usuario y contraseña), verifica su validez y retorna un JWT.
POST	<code>/auth/recover-password</code>	Inicia el proceso de recuperación de cuenta recibiendo un email. Genera y publica un evento <code>usuario.recuperacion</code> .
POST	<code>/auth/reset-password</code>	Recibe el token de activación/recuperación y la nueva contraseña para establecerla en la base de datos.
POST	<code>/auth/change-password</code>	Cambio de contraseña de un usuario autenticado. Recibe la contraseña actual y la nueva; valida la política de seguridad. Requiere JWT válido.

Integración por Eventos (RabbitMQ / Kafka)

El **sistema de autenticación debe integrarse al ecosistema de eventos** (creado en el Reto 4) para automatizar el ciclo de vida completo de las cuentas:

Evento a Consumir	Acción en el auth-service
empleado.creado	Crea la cuenta en estado inactivo, sin contraseña válida . Genera un Token de Activación (ver estructuras más abajo) y le asigna el rol <code>USER</code> . Inmediatamente después publica <code>usuario.creado</code> con el email y el token (nunca una contraseña en plano).
empleado.retirado	Desactiva la cuenta de forma permanente , impidiendo futuros logins. Publica <code>cuenta.desactivada</code> con motivo: <code>"RETIRO"</code> y <code>permanente: true</code> .
vacaciones.iniciadas	Desactiva la cuenta temporalmente . Publica <code>cuenta.desactivada</code> con motivo: <code>"VACACIONES"</code> y <code>permanente: false</code> .
vacaciones.finalizadas	Reactiva la cuenta solo si el empleado no fue retirado durante el período . Publica <code>cuenta.activada</code> con motivo: <code>"FIN_VACACIONES"</code> .

Caso borde obligatorio. Si un empleado es retirado mientras está de vacaciones, al llegar la fechaFin llegará `vacaciones.finalizadas` sobre una cuenta con baja permanente. El `auth-service` **no debe reactivarla**. Debe distinguir la desactivación temporal de la permanente en su modelo de datos (por ejemplo, un estado `ACTIVA | SUSPENDIDA_TEMPORAL | DESACTIVADA_PERMANENTE` , no un simple booleano). Este escenario se evalúa.

Eventos Salientes del auth-service

Evento a Publicar	Disparador	Consumido por
usuario.creado	Consumo de <code>empleado.creado</code>	<code>notificaciones-service</code>
usuario.recuperacion	Petición a <code>/auth/recover-password</code>	<code>notificaciones-service</code>

Evento a Publicar	Disparador	Consumido por
cuenta.activada	Establecimiento inicial de contraseña, o consumo de vacaciones.finalizadas	notificaciones-service
cuenta.desactivada	Consumo de vacaciones.iniciadas o de empleado.retirado	notificaciones-service

Las cargas útiles están definidas en el [Catálogo de Eventos](#), secciones 3.4 a 3.7.

Consumo en el notificaciones-service

El servicio de notificaciones construido en el Reto 4 debe actualizarse para escuchar los nuevos eventos emitidos por el auth-service . Al consumirlos, simulará el envío de un correo electrónico al empleado:

Para usuario.creado o usuario.recuperacion :

[NOTIFICACIÓN] Tipo: SEGURIDAD | Para: juan@empresa.com | Mensaje: "Para establecer o r

(Nota: En un escenario de producción, este log simularía el envío de un correo con un link del estilo <https://app.empresa.com/reset?token=xyz123>)

Para cuenta.desactivada / cuenta.activada :

[NOTIFICACIÓN] Tipo: CUENTA | Para: juan@empresa.com | Mensaje: "Su cuenta fue desactiv
[NOTIFICACIÓN] Tipo: CUENTA | Para: juan@empresa.com | Mensaje: "Bienvenido de regreso.

Importante: el correo de bienvenida del onboarding se dispara con usuario.creado , **no** con empleado.creado . Es usuario.creado el que trae el token de activación, y sin token el correo sería inútil.

Estructuras de Tokens (Acceso y Activación)

El sistema ahora manejará dos tipos de tokens con propósitos diferentes:

1. Token de Acceso (Access JWT)

Es el token que se retorna en `/auth/login`. Su payload debe tener, como mínimo, la siguiente estructura:

```
{
  "sub": "nombre_de_usuario",
  "role": "ADMIN | USER",
  "iat": 1712345678,
  "exp": 1712349278
}
```

*Nota: La contraseña no debe incluirse **nunca** en el token JWT, y en la base de datos debe almacenarse encriptada/hasheada (ej. BCrypt).*

2. Token de Recuperación/Establecimiento (Reset Token)

Es el token que se genera al crear un empleado o al solicitar recuperar la contraseña. Para su implementación, el equipo puede elegir una de las siguientes opciones:

Opción A (Recomendada - Stateless con JWT): Reusar la librería JWT para generar un token firmado específicamente para este propósito. Debe incluir un *claim* especial para distinguirlo de un token de acceso y un tiempo de expiración corto (ej. 15 minutos a 1 hora).

```
{
  "sub": "nombre_de_usuario",
  "type": "RESET_PASSWORD",
  "iat": 1712345678,
  "exp": 1712349278
}
```

Ventaja: El `auth-service` puede validar matemáticamente su autenticidad y expiración sin necesidad de consultar ni guardar el token en la base de datos.

Opción B (Stateful con UUID): Generar un UUID aleatorio (ej. `d3b07384-d9a7...`) y guardarlo en la base de datos asociado al usuario, junto con una fecha de expiración (`expiresAt`).

Ventaja: Es fácil de revocar inmediatamente si el usuario lo usa.

Desventaja: Requiere tablas adicionales en base de datos o almacenamiento en caché (Redis).

2. Protección de los Microservicios (Validación JWT)

Una vez que el `auth-service` puede emitir tokens, los demás servicios deben exigir este token en la cabecera HTTP `Authorization` usando el esquema `Bearer`.

Formato esperado en las peticiones

Los clientes deberán enviar el token en todas las peticiones a los recursos protegidos:

Authorization: Bearer <token_jwt_aqui>

Opciones de Implementación para la Validación (Elegir una)

1. **En el API Gateway (recomendado):** el Gateway construido en el **Reto 3** ya recibe todas las peticiones externas. Agregarle la validación del JWT significa escribirla **una sola vez**: intercepta la solicitud, verifica la firma, y solo entonces la enruta al microservicio correspondiente.
2. **Middleware/Interceptor por Servicio:** Configurar cada microservicio individual (Empleados, Departamentos, Perfiles, Vacaciones...) con una librería de seguridad que intercepte y valide la firma del JWT antes de procesar el controlador.

Por qué la primera opción es claramente mejor aquí. El ecosistema ya tiene seis servicios en cuatro lenguajes distintos. La opción 2 implica resolver la validación de JWT **cuatro veces**, con cuatro librerías diferentes, y mantenerlas sincronizadas. Además, el **Reto 10** traslada la validación a JWKS contra un Identity Provider: si la lógica está repartida por seis servicios, ese cambio hay que hacerlo seis veces.

Este es exactamente el problema que el Gateway del Reto 3 se anticipó a resolver. La opción 2 sigue disponible, pero si la eligen, **justifiquen por qué** aceptan ese costo.

Reglas de Autorización (RBAC)

Debe aplicar las siguientes reglas utilizando la información del token:

- **RoI ADMIN** : Tiene acceso total. Puede crear, modificar y eliminar recursos en cualquier servicio. (ej. `DELETE /empleados/{id}` , `POST /departamentos` , `POST /vacaciones`).
- **RoI USER** : Acceso de solo lectura sobre los recursos del ecosistema (ej. `GET /empleados` , `GET /perfiles/{id}`), **con dos excepciones sobre sus propios datos**:
 - `POST /auth/change-password` — puede cambiar su propia contraseña.
 - `PUT /perfiles/{empleadoId}` — puede actualizar **su propio** perfil, y solo el suyo.

- **Peticiones sin Token válido:** Deben recibir un código HTTP 401 Unauthorized .
- **Peticiones de un rol sin permisos:** Deben recibir un código HTTP 403 Forbidden .

Autorización por propiedad del recurso

El RBAC por rol no alcanza para el caso anterior: un `USER` no puede escribir en general, pero sí sobre lo suyo. Esto requiere una segunda comprobación, sobre la **propiedad del recurso**:

```

si rol == ADMIN          → permitir
si rol == USER y método es de lectura → permitir
si rol == USER y recurso.EmpleadoId == token.sub → permitir
en cualquier otro caso   → 403 Forbidden

```

El `EmpleadoId` del recurso se compara contra el `claim sub` del token. Un `USER` que intente modificar el perfil de otro empleado debe recibir 403 Forbidden , **no** 401 .

Este es el primer punto del curso donde la autorización deja de depender solo del rol y pasa a depender también de **quién es dueño del dato**. Es la diferencia entre RBAC y ABAC (*Attribute-Based Access Control*), y es lo que hace posible que cada empleado gestione su propio perfil sin darle permisos de administrador.

3. Scheduler de Vacaciones

En el Reto 4 el `vacaciones-service` quedó a medias: registra períodos y publica `vacaciones.programadas` , pero **nada ocurre cuando llega la fecha de inicio**. La cuenta del empleado nunca se desactiva porque no hay nada que observe el paso del tiempo. Es hora de terminar con esa funcionalidad.

3.1 Comportamiento requerido

El `vacaciones-service` debe ejecutar una tarea periódica que:

1. Busque los períodos en estado `PROGRAMADA` cuya `fechaInicio` ya llegó
→ los pasa a `EN_CURSO` y publica `vacaciones.iniciadas` .
2. Busque los períodos en estado `EN_CURSO` cuya `fechaFin` ya pasó
→ los pasa a `FINALIZADA` y publica `vacaciones.finalizadas` .

La frecuencia sugerida es **cada minuto** durante el desarrollo (para poder probar sin esperar un día) y se documenta como configurable por variable de entorno.

3.2 Implementación por lenguaje

Lenguaje / Framework	Mecanismo
Java / Spring Boot	@Scheduled(cron = "\${VACACIONES_CRON}") + @EnableScheduling
Node.js	node-cron o setInterval
Python	APScheduler , o BackgroundTasks con bucle asíncrono
Go	time.Ticker en una goroutine, o robfig/cron
.NET	IHostedService / BackgroundService con PeriodicTimer

3.3 Cómo probarlo sin esperar

Para demostrar el flujo completo en la sustentación, el período debe poder dispararse en segundos. Elija y documente una de las siguientes estrategias con el fin de poder realizar las pruebas y tomar evidencias que muestren el correcto funcionamiento del sistema:

- Programar vacaciones con `fechaInicio = hoy` y `fechaFin = hoy`, con el scheduler corriendo cada minuto.
- Permitir que las fechas se manejen con precisión de **fecha y hora** (no solo día) en el entorno de desarrollo.
- Exponer un endpoint de prueba `POST /vacaciones/{id}/forzar-inicio` que dispare el evento manualmente, **claramente marcado como endpoint de desarrollo** y protegido con rol `ADMIN`.

3.4 La limitación que este diseño arrastra

⚠ **Léalo con atención: esto se rompe en el Reto 11.**

Un `@Scheduled` (o equivalente) se ejecuta **en cada instancia del servicio**. Mientras haya una sola instancia, funciona. En el Reto 11 el ecosistema escala horizontalmente, y entonces **las N instancias del `vacaciones-service` dispararán el mismo evento N veces**: N correos, N desactivaciones.

Hay dos defensas, y conviene entender que son complementarias:

1. **Deduplicación en el consumidor** (Reto 4): el `auth-service` descarta el evento repetido. Mitiga el daño, pero no evita el trabajo duplicado.
2. **Coordinación en el productor**: garantizar que solo una instancia ejecute el job. Esto es lo que resuelve el **Reto 31** con **ShedLock** (un lock distribuido en base de datos).

En este reto se acepta la versión de instancia única. **Documente la limitación en el README**; el Reto 11 la usará como caso de estudio de estado en servicios escalados.

4. Pruebas del Sistema

Verificar el funcionamiento completo del flujo de seguridad:

Flujo de prueba sugerido

1. Iniciar todos los servicios: `docker-compose up --build`
2. **Onboarding de empleado**: Petición `POST /empleados` (abierto por defecto sin token o simular una creación directa si el endpoint ya está protegido para que el Admin inicial lo cree).
Nota: Generalmente se requiere un usuario Administrador "semilla" (seed) creado por base de datos o variable de entorno al levantar el servicio.
3. **Verificar Eventos**: Confirmar en los logs del `auth-service` que se detectó `empleado.creado` y se generó el usuario. Luego, confirmar en los logs de `notificaciones-service` que se detectó `usuario.creado` y se simuló el envío de credenciales.
4. **Petición denegada**: Intentar acceder a `GET /empleados` sin cabeceras. (Debe fallar con `401 Unauthorized`).
5. **Establecer Contraseña (Reset)**: Extraer el token de recuperación de los logs de la notificación. Enviar una petición `POST /auth/reset-password` con el token y la nueva contraseña deseada.
6. **Login USER**: Realizar login en `/auth/login` utilizando el correo del empleado recién creado y la contraseña que acaba de establecer. Extraer el token JWT de la respuesta.
7. **Lectura exitosa**: Hacer una petición a `GET /empleados` enviando el JWT en la cabecera `Authorization: Bearer <token>`. (Debe retornar `200 OK`).
8. **Prueba de Recuperación**: Enviar `POST /auth/recover-password` con el email del usuario. Verificar en los logs de notificaciones que se generó un nuevo evento `usuario.recuperacion` con un nuevo token. Repetir el paso 5 con este nuevo token y el paso 6 para validar el cambio.
9. **Escritura denegada (403)**: Intentar hacer `DELETE /empleados/E001` con el JWT del usuario regular. (Debe fallar con `403 Forbidden`).

10. **Propiedad del recurso:** Con el JWT de E001 , hacer PUT /perfiles/E001 (debe retornar 200 OK) y luego PUT /perfiles/E002 (debe fallar con 403 Forbidden).
11. **Cambio de contraseña:** Con el JWT de E001 , enviar POST /auth/change-password con la contraseña actual y una nueva. Verificar que el login con la contraseña antigua falla y con la nueva funciona.

Flujo de vacaciones (nuevo)

12. **Programar vacaciones:** Con el JWT del ADMIN , POST /vacaciones para E001 con fechaInicio = hoy y fechaFin = hoy. Verificar que se publicó vacaciones.programadas y que llegó la notificación de confirmación.
13. **Desactivación automática:** Esperar a que corra el scheduler (o forzar el inicio) y verificar, en este orden:
 - En los logs del vacaciones-service : se publicó vacaciones.iniciadas .
 - En los logs del auth-service : consumió el evento y suspendió la cuenta.
 - En los logs del notificaciones-service : llegó cuenta.desactivada con motivo: VACACIONES .
 - POST /auth/login con las credenciales de E001 ahora falla (401).
14. **Reactivación automática:** Esperar a que el scheduler detecte la fechaFin y verificar que se publicó vacaciones.finalizadas , que la cuenta se reactivó (cuenta.activada con motivo: FIN_VACACIONES) y que POST /auth/login vuelve a funcionar.
15. **Caso borde — retiro durante vacaciones:** Programar vacaciones para E002 , dejar que inicien, y retirar al empleado mientras está de vacaciones (DELETE /empleados/E002 con JWT de ADMIN). Al llegar la fechaFin , verificar que vacaciones.finalizadas **NO reactiva la cuenta:** el login sigue fallando.

Offboarding

16. **Offboarding (Admin):** Con el JWT del administrador "semilla", enviar DELETE /empleados/E001 . (Debe retornar 200 OK o 204 No Content , cambiar el estado a RETIRADO y publicar empleado.retirado).
17. **Verificar Inhabilitación:** Intentar hacer login nuevamente con las credenciales del empleado retirado. (Debe fallar con 401 Unauthorized o 403 Forbidden).
18. **Verificar auditoría:** GET /empleados?estado=RETIRADO debe listar al empleado con su fechaRetiro .

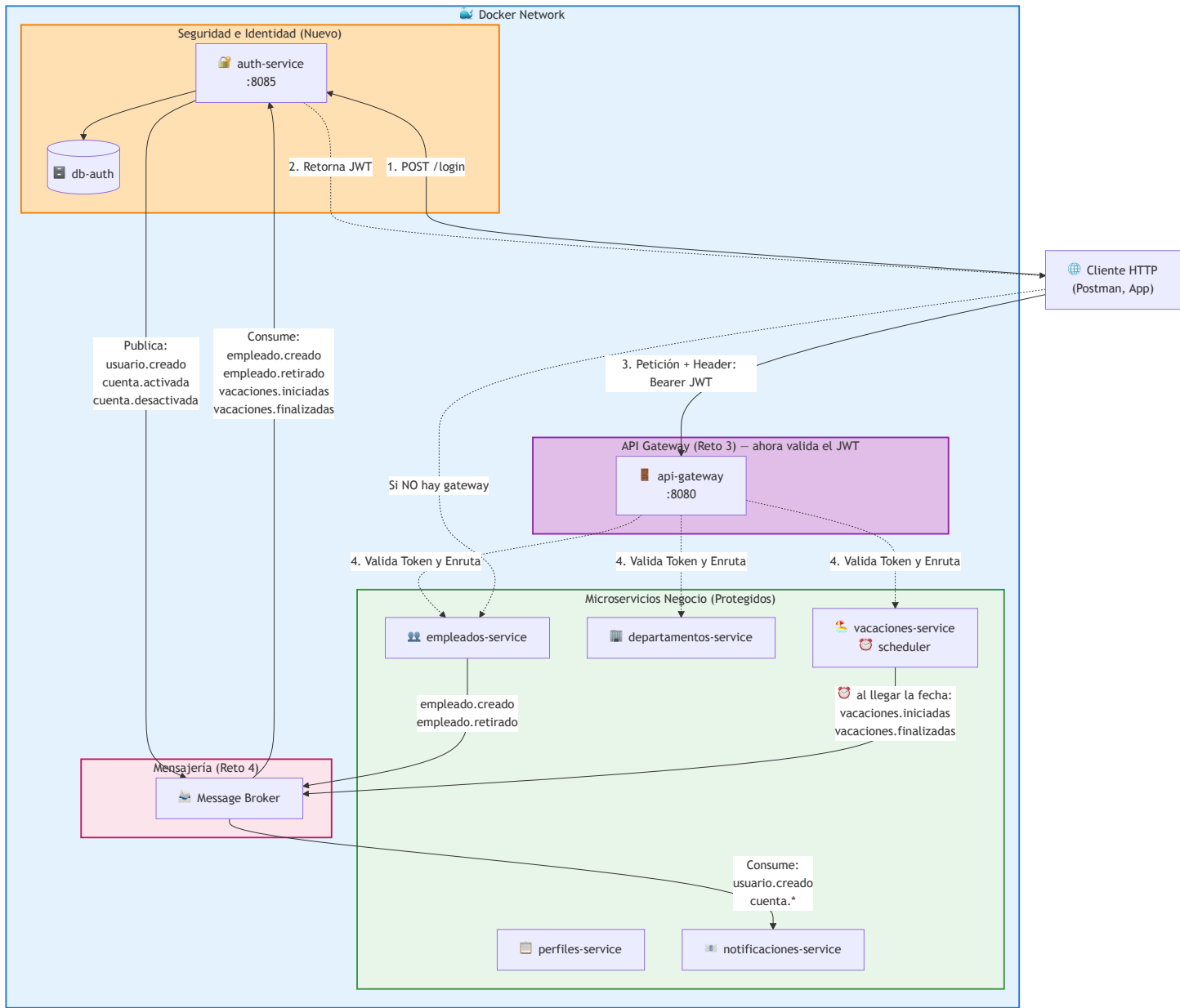
Entregables

- Código versionado en sus respectivos repositorios de GitHub.
- Configuración de `docker-compose.yml` actualizada que incluya el nuevo `auth-service` y, si se usa, el componente API Gateway.
- Modificar el [README.md](#) para incluir:
 - Instrucciones de cómo obtener un token para realizar pruebas locales (ej. colección de Postman/Bruno).
 - Explicación de qué estrategia de validación de token se seleccionó (Gateway vs. Interceptores individuales) y por qué.
 - La clave secreta (secret key) del JWT (únicamente por propósitos académicos) documentada o definida claramente en el `.env.example`.
 - **Diagrama de secuencia del ciclo de vida de la cuenta:** desde `empleado.creado` hasta el retiro, pasando por la suspensión y reactivación por vacaciones.
 - **Mecanismo y frecuencia del scheduler** elegido, cómo se configura, y la estrategia usada para demostrar el flujo sin esperar días reales.
 - **Limitación conocida del scheduler:** qué ocurre si el `vacaciones-service` se escala a N instancias, y cómo se resolverá (Reto 31 / ShedLock).
 - Evidencia del **caso borde**: empleado retirado durante sus vacaciones cuya cuenta no se reactiva al finalizar el período.

Consideraciones

- Tanto el `auth-service` (que firma el token) como los servicios que lo validan deben usar el **mismo "Secret"** o clave simétrica. En un entorno de microservicios real con clave asimétrica, compartirían la clave pública, pero por simplicidad académica, una firma simétrica (HMAC SHA-256) es suficiente.
- El secreto debe inyectarse a todos los contenedores a través de **variables de entorno** en Docker Compose.
- Asegúrese de que la UI de Swagger esté configurada con el esquema de seguridad *BearerAuth* para permitir probar las APIs cómodamente sin usar 100% herramientas como Postman.

Diagrama de Arquitectura Esperada



Criterios de Evaluación

El reto se evalúa sobre **5 puntos**, distribuidos de la siguiente forma:

#	Elemento	Valor	Aspectos a evaluar
1	Servicio de Autenticación	1.0	Creación del auth-service , almacenamiento seguro de credenciales (hash), emisión correcta de tokens con formato estándar JWT. Token de activación con expiración;

#	Elemento	Valor	Aspectos a evaluar
			nunca una contraseña en plano. Endpoints <code>/login</code> , <code>/recover-password</code> , <code>/reset-password</code> y <code>/change-password</code> funcionales.
2	Validación de Token	0.5	Los servicios exigen el JWT y validan su firma correctamente. Se rechazan peticiones sin token o con token alterado (<code>401</code>).
3	Control de Acceso (RBAC + propiedad del recurso)	1.0	Se respetan los roles <code>ADMIN</code> y <code>USER</code> (<code>403</code> en acciones no autorizadas). Un <code>USER</code> puede actualizar su propio perfil y cambiar su contraseña, y recibe <code>403</code> al intentar modificar los datos de otro empleado.
4	Ciclo de vida de la cuenta por eventos + scheduler	1.5	El <code>auth-service</code> consume <code>empleado.creado</code> , <code>empleado.retirado</code> , <code>vacaciones.iniciadas</code> y <code>vacaciones.finalizadas</code> , y publica los cuatro eventos salientes según el catálogo. Scheduler funcional en <code>vacaciones-service</code> que dispara inicio y fin de período. Flujo completo demostrado: login OK → vacaciones inician → login falla → vacaciones terminan → login OK. Caso borde del retiro durante vacaciones resuelto correctamente.
5	Gestión de variables de entorno	0.5	El "Secret" del JWT y otras configuraciones se inyectan vía <code>docker-compose.yml</code> a los contenedores necesarios, y no están "quemadas" (hardcoded) en el código.
6	Documentación e Integración	0.5	Sistema completo levantable con <code>docker compose up</code> , OpenAPI (Swagger) soportando el esquema <code>BearerAuth</code> , diagrama de secuencia del ciclo de vida de la cuenta, y limitación del scheduler documentada.